

CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks

Q2
Customer Vibe Analysis
— Set + List
10 marks

Q3
Product Inventory Analysis
— List + Tuple
10 marks

Q4
Tune Analysis
— Set + List
10 marks

Q5
Employee Self Analysis
— Tuple + List + String
10 marks

Q6
Transaction Analysis
— Tuple + List
10 marks

Q7
Network Security Analysis
— Set + List
10 marks

Q8
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9
Web Frequency and Date
Analysis
— Set + List + Tuple
10 marks

Q10
Integrated Student Data
Analysis
— Set + List + Tuple
10 marks

10/10 answered

Q1: Student Performance Analysis — List + Tuple

10 marks

A college stores the following student records:

```
students = [
    ("Arun", [78, 85, 92, 67]),
    ("Bala", [45, 67, 55, 49]),
    ("Charan", [90, 90, 88, 95]),
    ("Divya", [65, 72, 70, 68]),
    ("Esha", [88, 76, 91, 84])
]
```

Analyze the student data and perform the following operations:

1. Calculate the total and average marks of every student.
2. Identify the student with the highest total.
3. Identify the student with the lowest average.
4. Find students whose average is greater than the class average.
5. Find students who scored above 80 in at least three subjects.
6. Find the highest mark in each subject.
7. Identify students who scored below 50 in any subject.
8. Sort the students based on their average marks in descending order.
9. Identify the top two students.
10. Explain which parts of the structure are mutable and which are immutable.

Expected Output

Student Totals:

Arun : 322

Bala : 216

Charan : 365

Divya : 275

Esha : 339

Student Averages:

Arun : 80.50

Bala : 54.00

Charan : 91.25

Divya : 68.75

Esha : 84.75

Highest Total: Charan

Lowest Average: Bala

Above Class Average:

Arun, Charan, Esha

Students with 1 or more marks above 80:

Charan

Subject-wise Highest

Subject 1: 90

Subject 2: 92

Subject 3: 92

Subject 4: 95

Students with a mark below 50:

Bala

Top 2:

Charan

Esha

Your Code

```
1 students = [
2     ("Arun", [78, 85, 92, 67]),
3     ("Bala", [45, 67, 55, 49]),
4     ("Charan", [90, 90, 88, 95]),
5     ("Divya", [65, 72, 70, 68]),
6     ("Esha", [88, 76, 91, 84])
7 ]
8
9
10 totals = {name: sum(marks) for name, marks in students}
11 averages = {name: sum(marks)/len(marks) for name, marks in students}
12
13 print("Student totals:")
14 for name, total in totals.items():
15     print(f"{name} : {total}")
16
17 print("\nStudent averages:")
18 for name, avg in averages.items():
19     print(f"{name} : {avg:.2f}")
20
21 print(f"\nhighest total: {max(totals, key=totals.get)}")
22 print(f"lowest average: {min(averages, key=averages.get)}")
23
24 class_avg = sum(averages.values()) / len(averages)
25 above_avg = [name for name, avg in averages.items() if avg > class_avg]
26 print(f"above Class Average:")
27 for name in above_avg:
28     print(name)
```

Input

Output

Student totals:

Arun : 322

Bala : 216

Charan : 365

Divya : 275

Esha : 339

CO4: AT2 CASE STUDY

Back

QUESTIONS

Q1

Student Performance Analysis
— List + Tuple
10 marks

Q2

Customer Name Analysis —
String + List
10 marks

Q3

Product Inventory Analysis —
List + Tuple
10 marks

Q4

Test Analysis — String + List
10 marks

Q5

Employee Skill Analysis —
Tuple + List + String
10 marks

Q6

Transaction Analysis — Tuple +
List
10 marks

Q7

Password Security Analysis —
String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Web Request and Data
Analysis — String + List +
Tuple
10 marks

Q10

Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Customer Name Analysis — String + List

10 marks

An e-commerce company receives customer names in inconsistent formats:

```
names = [
```

```
    "Arun Kumar",
```

```
    "ARUN KUMAR",
```

```
    "arun kumar",
```

```
    "Bala Krishnan",
```

```
    "BALA KRISHNAN",
```

```
    "Charan",
```

```
    "charan",
```

```
    "Divya Devi",
```

```
    "Divya Devi"
```

```
]
```

Analyze the customer data.

Tasks

1. Remove unnecessary leading and trailing spaces.
2. Convert all names to lowercase.
3. Identify duplicate customers.
4. Count the number of unique customers.
5. Find customers whose names contain more than one word.
6. Find the longest customer name.
7. Find customers whose names start with 'a' or 'd'.
8. Count the number of characters in each normalized name.
9. Sort the unique names alphabetically.
10. Determine which customer name occurs most frequently.

Expected Output:

Normalized:

```
['arun kumar', 'arun kumar', 'arun kumar',
```

```
 'bala krishnan', 'bala krishnan',
```

```
 'charan', 'charan',
```

```
 'divya devi', 'divya devi']
```

Unique Customers:

```
['arun kumar', 'bala krishnan', 'charan', 'divya devi']
```

Number of Unique Customers: 4

Duplicate Customers:

```
arun kumar
```

```
bala krishnan
```

```
charan
```

```
divya devi
```

Names with Multiple Words:

```
arun kumar
```

```
bala krishnan
```

```
divya devi
```

Longest Name:

```
bala krishnan
```

Starts with A or D:

```
arun kumar
```

```
divya devi
```

Your Code

```
1 names = [
2     "Arun Kumar",
3     "ARUN KUMAR",
4     "arun kumar",
5     "Bala Krishnan",
6     "BALA KRISHNAN",
7     "Charan",
8     "charan",
9     "Divya Devi",
10    "divya devi"
11 ]
12
13 normalized = [name.strip().lower() for name in names]
14
15 unique_customers = sorted(list(set(normalized)))
16
17 duplicates = []
18 for name in unique_customers:
19     if normalized.count(name) > 1:
20         duplicates.append(name)
21
22 multi_word = [name for name in unique_customers if len(name.split()) >
23               1]
24 longest_name = max(unique_customers, key=len)
25
26 starts_ad = [name for name in unique_customers if name.startswith('a')
27               or name.startswith('d')]
```

Input

```
names = [
    "Arun Kumar",
    "ARUN KUMAR",
    "arun kumar",
```

Output

CO4: AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
30 marks**Q2**
Customer Service Analysis
— String + List
30 marks**Q3**
Product Inventory Analysis
— List + Tuple
30 marks**Q4**
Risk Analysis — String + List
30 marks**Q5**
Employee Skill Analysis
— Tuple + List + String
30 marks**Q6**
Transaction Analysis — Tuple + List
30 marks**Q7**
Network Security Analysis
— String + List
30 marks**Q8**
Code Analysis and Debugging
— List + Tuple + String
30 marks**Q9**
Email Frequency and Date Analysis
— String + List + Tuple
30 marks**Q10**
Integrated Student Data Analysis
— String + List + Tuple
30 marks

10/10 answered

Q3 Product Inventory Analysis — List + Tuple

10 marks

A supermarket stores product information as:

```
products = [  
    ("Laptop", 5, 55000),  
    ("Mouse", 25, 700),  
    ("Keyboard", 8, 1200),  
    ("Monitor", 12, 15000),  
    ("Printer", 4, 12000),  
    ("Headset", 15, 2500),  
]
```

Each tuple contains:

(Product Name, Quantity, Price)

Analyze the inventory:

tasks

1. Calculate the inventory value of every product.
2. Calculate the total inventory value.
3. Identify products whose quantity is less than 10.
4. Identify products whose price is greater than 150,000.
5. Find the most expensive product.
6. Find the product having the highest inventory value.
7. Sort products based on price.
8. Sort products based on quantity.
9. Calculate the total quantity of all products.
10. Identify products contributing more than 150,000 to inventory value.

Expected Output:

Inventory Values:

Laptop : 275000

Mouse : 17500

Keyboard : 9600

Monitor : 180000

Printer : 48000

Headset : 17500

Total Inventory Value: 567500

Low Stock:

Laptop

Keyboard

Printer

Price > 100000

Laptop

Monitor

Printer

Most expensive:

Laptop

Highest Inventory Value:

Laptop

Total Quantity: 69

Your Code

```
1 products = [  
2     ("Laptop", 5, 55000),  
3     ("Mouse", 25, 700),  
4     ("Keyboard", 8, 1200),  
5     ("Monitor", 12, 15000),  
6     ("Printer", 4, 12000),  
7     ("Headset", 15, 2500)  
8 ]  
9  
10 inventory_values = {}  
11 total_inventory_value = 0  
12  
13 for name, qty, price in products:  
14     value = qty * price  
15     inventory_values[name] = value  
16     total_inventory_value += value  
17  
18 print("Inventory values:")  
19 for name, value in inventory_values.items():  
20     print(f"{name} : {value}")  
21  
22 print(f"\nTotal inventory value: {total_inventory_value}")  
23  
24 low_stock = [name for name, qty, price in products if qty < 10]  
25 print("\nLow Stock:")  
26 for name in low_stock:  
27     print(name)  
28  
29 price_gt_100000 = [name for name, qty, price in products if price > 100000]  
30 print("\nPrice > 100000:")  
31 for name in price_gt_100000:  
32     print(name)
```

Input

```
products = [  
    ("Laptop", 5, 55000),  
    ("Mouse", 25, 700),  
    ("Keyboard", 8, 1200),  
]
```

Output

```
Inventory values:  
Laptop : 275000  
Mouse : 17500  
Keyboard : 9600  
Monitor : 180000  
Printer : 48000  
Headset : 17500
```


CO4. AT2. CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks**Q2**
Customer Name Analysis
— String + List
10 marks**Q3**
Product Inventory Analysis
— List + Tuple
10 marks**Q4**
Text Analysis — String + List
10 marks**Q5**
Employee Skill Analysis
— Tuple + List + String
10 marks**Q6**
Transaction Analysis — Tuple + List
10 marks**Q7**
Password Security Analysis
— String + List
10 marks**Q8**
Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**
Word Frequency and Data Analysis
— String + List + Tuple
10 marks**Q10**
Integrating Student Data Analysis
— String + List + Tuple
10 marks

10/10 answered

Q4 Text Analysis — String + List

10 marks

A university receives the following feedback:

feedback = ""

Python programming is easy to learn.

Python programming is powerful.

Learning Python requires practice.

Practice makes programming skills better.

—

Analyze the text

tasks

1. Convert the entire text to lowercase.
2. Remove punctuation.
3. Convert the text into a list of words.
4. Count the total number of words.
5. Find the number of unique words.
6. Find the frequency of "python".
7. Find the frequency of "programming".
8. Identify words appearing more than once.
9. Find the longest word.
10. Sort the unique words alphabetically.
11. Find all words containing the letter 'p'.
12. Find the percentage of unique words.

Expected Output:

Total Words: 21

Unique Words: 16

Frequency of python: 3

Frequency of programming: 3

Repeated Words:

python

programming

is

learning

practice

Longest Word:

programming

Words containing 'p':

python

programming

powerful

practice

Your Code

```
1 import string
2
3 feedback = """
4 python programming is easy to learn.
5 python programming is powerful.
6 learning python requires practice.
7 practice makes programming skills better.
8 """
9
10 # 1. Convert the entire text to lowercase
11 text = feedback.lower()
12
13 # 2. remove punctuation
14 text = text.translate(str.maketrans('', '', string.punctuation))
15
16 # 3. Convert the text into a list of words
17 words = text.split()
18
19 # 4. Count the total number of words
20 total_words = len(words)
21
22 # 5. Find the number of unique words
23 unique_words_set = set(words)
24 num_unique_words = len(unique_words_set)
25
26 # 6. Find the frequency of "python"
27 freq_python = words.count("python")
28
29 # 7. Find the frequency of "programming"
30 freq_programming = words.count("programming")
31
32 # 8. Identify words appearing more than once (preserve first appearance)
33 repeated_words = []
34 for w in words:
35     if words.count(w) > 1 and w not in repeated_words:
36         repeated_words.append(w)
37
```

Input

Python programming is easy to learn.
Python programming is powerful.
Learning Python requires

Output

total words: 21
unique words: 16

frequency of python: 3
frequency of programming: 3

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1
Structure Performance Analysis
— List + Tuple
10 marks

Q2
Customer Name Analysis
— String + List
10 marks

Q3
Product Inventory Analysis
— List + Tuple
10 marks

Q4
Data Analysis — String + List
10 marks

Q5
Employee Skill Analysis —
Tuple + List + String
10 marks

Q6
Simulation Analysis — Tuple +
List
10 marks

Q7
Forecast Security Analysis —
String + List
10 marks

Q8
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9
Word Frequency and Data
Analysis — String + List +
Tuple
10 marks

Q10
Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

05 Employee Skill Analysis — Tuple + List + String

10 marks

An organization stores:

employees = [

("E101", "Arun", ["Python", "SQL", "ML"]),

("E102", "Bala", ["Java", "SQL", "HTML"]),

("E103", "Charan", ["Python", "ML", "SQL"]),

("E104", "Divya", ["Python", "Java", "Cloud"]),

("E105", "Esha", ["Python", "ML", "Cloud"])]

]

Analyze the employee skill data.

Tasks

1. Find all employees who know Python.
2. Find employees who know both Python and ML.
3. Find employees who know SQL but not Java.
4. Find employees who know either Java or Cloud.
5. Count employees having each skill.
6. Identify the most common skill.
7. Identify employees having at least three skills.
8. Find employees who do not know Python.
9. Sort employees based on the number of skills.
10. Determine which employees are suitable for a Python + ML project.

Expected Output:

Python:

Arun, Charan, Divya, Esha

Python + ML:

Arun, Charan, Esha

SQL but not Java:

Arun, Charan

Java or Cloud:

Bala, Divya, Esha

Employees with 3 skills:

Arun, Bala, Charan, Divya, Esha.

Without Python:

Bala

Most Common Skill:

Python

Suitable for Python + ML:

Arun, Charan, Esha

Your Code

```
1 employees = [
2     ("E101", "Arun", ["Python", "SQL", "ML"]),
3     ("E102", "Bala", ["Java", "SQL", "HTML"]),
4     ("E103", "Charan", ["Python", "ML", "SQL"]),
5     ("E104", "Divya", ["Python", "Java", "Cloud"]),
6     ("E105", "Esha", ["Python", "ML", "Cloud"])
7 ]
8
9 python_knowers = [emp[1] for emp in employees if "python" in emp[2]]
10 python_ml_knowers = [emp[1] for emp in employees if "python" in emp[2]]
11
12 sql_no_java = [emp[1] for emp in employees if "SQL" in emp[2] and "java" not in emp[2]]
13
14 java_or_cloud = [emp[1] for emp in employees if "java" in emp[2] or "Cloud" in emp[2]]
15
16 skill_counts = {}
17 for emp in employees:
18     for skill in emp[2]:
19         skill_counts[skill] = skill_counts.get(skill, 0) + 1
20
21 most_common_skill = max(skill_counts, key=skill_counts.get)
22 three_skills = [emp[1] for emp in employees if len(emp[2]) >= 3]
23
24 no_python = [emp[1] for emp in employees if "python" not in emp[2]]
25
26 sorted_employees = sorted(employees, key=lambda x: len(x[2]))
27
28 suitable_pm = [emp[1] for emp in employees if "python" in emp[2] and "ML" in emp[2]]
29
30 print("python:")
31 print(", ".join(python_knowers))
32 print("\npython + ml:")
33 print(", ".join(python_ml_knowers))
34 print("\nSQL but not java:")
35 print(", ".join(sql_no_java))
36 print("\njava or Cloud:")
37 print(", ".join(java_or_cloud))
38 print("\nemployees with 3 skills:")
```

Input

```
employees = [
    ("E101", "Arun", ["Python",
    "SQL", "ML"]),
    ("E102", "Bala", ["Java",
```

Output

```
python:
Arun, Charan, Divya, Esha

python + ml:
Arun, Charan, Esha
```

CO4. AT2 CASE STUDY

← Back

QUESTIONS

Q1

Student Performance Analysis —
→ List + Tuple
10 marks

Q2

Customer Name Analysis —
String + List
10 marks

Q3

Product Inventory Analysis —
List + Tuple
10 marks

Q4

Test Analysis — String + List
10 marks

Q5

Employee Skill Analysis —
Tuple + List + String
10 marks

Q6

Transaction Analysis — Tuple +
List
10 marks

Q7

Network Security Analysis —
String + List
10 marks

Q8

Crime Analysis and Debussing —
List + Tuple + String
10 marks

Q9

Hired Frequency and Date
Analysis — String + List +
Tuple
10 marks

Q10

Integrated Database Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Q6 Transaction Analysis — Tuple + List

10 marks

A bank stores transactions:

transactions = [

("T101", "Deposit", 5000),

("T102", "Withdraw", 2000),

("T103", "Deposit", 8000),

("T104", "Withdraw", 1500),

("T105", "Deposit", 3000),

("T106", "Withdraw", 7000)

]

Analyze the transaction history.

Tasks

1. Calculate total deposits.
2. Calculate total withdrawals.
3. Calculate the net balance.
4. Find the largest deposit.
5. Find the largest withdrawal.
6. Identify all transactions greater than 40,000.
7. Count deposits and withdrawals.
8. Find the average transaction amount.
9. Sort transactions based on amount.
10. Identify whether total withdrawals exceed 50% of total deposits.

Expected Output:

Total Deposit: 16000

Total Withdrawal: 10500

Net Balance: 5500

Largest Deposit: 8000

Largest Withdrawal: 7000

Transactions > 4000:

T101

T103

T106

Deposits: 3

Withdrawals: 3

Average Transaction: 4416.67

Withdrawals > 50% of Deposits:

Yes

Your Code

```
1 transactions = [  
2     ("T101", "Deposit", 5000),  
3     ("T102", "Withdraw", 2000),  
4     ("T103", "Deposit", 8000),  
5     ("T104", "Withdraw", 1500),  
6     ("T105", "Deposit", 3000),  
7     ("T106", "Withdraw", 7000)  
8 ]  
9  
10 total_deposit = sum([c[2] for t in transactions if t[1] == "Deposit"])  
11 total_withdrawal = sum([c[2] for t in transactions if c[1] == "Withdrawal"])  
12  
13 net_balance = total_deposit - total_withdrawal  
14  
15 largest_deposit = max([c[2] for t in transactions if t[1] == "Deposit"])  
16 largest_withdrawal = max([c[2] for t in transactions if t[1] == "Withdrawal"])  
17  
18 greater_than_4000 = [c[0] for t in transactions if c[2] > 4000]  
19  
20 deposit_count = len([t for t in transactions if t[1] == "Deposit"])  
21 withdrawal_count = len([t for t in transactions if t[1] == "Withdrawal"])  
22  
23 avg_transaction = sum([c[2] for t in transactions]) / len(transactions)  
24  
25 sorted_transactions = sorted(transactions, key=lambda x: x[2])  
26  
27 withdrawals_exceed_50 = "yes" if total_withdrawal > (0.5 * total_deposit)  
28  
29 print(f"Total deposit: {total_deposit}")  
30 print(f"Total withdrawal: {total_withdrawal}")  
31 print(f"Net balance: {net_balance}")  
32 print()  
33 print(f"Largest deposit: {largest_deposit}")  
34 print(f"Largest withdrawal: {largest_withdrawal}")  
35 print()  
36 print("Transactions > 4000:")  
37 for t_id in greater_than_4000:  
38     print(t_id)  
39 print()  
40 print(f"Deposits: {deposit_count}")
```

Input

```
transactions = [  
("T101", "Deposit", 5000),  
("T102", "Withdraw", 2000),  
("T103", "Deposit", 8000),
```

Output

```
total deposit: 16000  
total withdrawal: 10500  
net balance: 5500  
  
largest deposit: 8000  
largest withdrawal: 7000
```


CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1

Student Performance Analysis
→ List + Tuple
10 marks

Q2

Customer Name Analysis
→ String + List
10 marks

Q3

Product Inventory Analysis
→ List + Tuple
10 marks

Q4

Top Analysts → String + List
10 marks

Q5

Employee Skill Analysis
→ Tuple + List + String
10 marks

Q6

Transaction Analysis → Tuple + List
10 marks

Q7

Password Security Analysis
→ String + List
10 marks

Q8

Code Analysis and Debugging
→ List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis → String + List + Tuple
10 marks

Q10

Integrated Student Data
Analysis → String + List + Tuple
10 marks

10/10 answered

Q7 Password Security Analysis — String + List

10 marks

A system received:

```
passwords = [
    "Python@123",
    "python123",
    "PYTHON@123",
    "Py@45",
    "Python123",
    "Java#4567",
    "Admin@2026"
]
```

A valid password must have:

- At least 8 characters
- Uppercase
- Lowercase
- Digit
- Special character

Analyze:

1. Check every password against all five conditions.
2. Identify valid passwords.
3. Identify invalid passwords.
4. For each invalid password, identify the failed conditions.
5. Count valid and invalid passwords.
6. Find the longest password.
7. Find passwords containing "python" irrespective of case.
8. Sort passwords based on length.
9. Determine the percentage of valid passwords.

Expected Output:

```
Valid
Python@123
Java#4567
Admin@2026
```

Invalid:

```
python123 → No uppercase, No special character
PYTHON@123 → No lowercase
Py@45 → Less than 8 characters
Python123 → No special character
```

Valid Count: 3

Invalid Count: 4

Longest Password:

Python@123

Python-related passwords:

```
Python@123
python123
PYTHON@123
Python123
```

Valid Percentage: 42.85%

Your Code

```
1 passwords = [
2     "Python@123",
3     "python123",
4     "PYTHON@123",
5     "Py@45",
6     "Python123",
7     "Java#4567",
8     "Admin@2026"
9 ]
10
11 valid_passwords = []
12 invalid_passwords = []
13
14 for pwd in passwords:
15     failed_reasons = []
16
17     if len(pwd) < 8:
18         failed_reasons.append("less than 8 characters")
19
20     if not any(char.isupper() for char in pwd):
21         failed_reasons.append("no uppercase")
22
23     if not any(char.islower() for char in pwd):
24         failed_reasons.append("no lowercase")
25
26     if not any(char.isdigit() for char in pwd):
27         failed_reasons.append("no digit")
28
29     if not any(not char.isalnum() for char in pwd):
```

Input

```
passwords = [
    "Python@123",
    "python123",
    "PYTHON@123",
    "Py@45",
    "Python123",
    "Java#4567",
    "Admin@2026"
]
```

Output

```
valid:
Python@123
Java#4567
Admin@2026
```

```
invalid:
```

CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
→ List + Tuple
10 marks

Q2
Customer Status Analysis
→ String + List
10 marks

Q3
Product Inventory Analysis
→ List + Tuple
10 marks

Q4
Text Analysis
→ String + List
10 marks

Q5
Employee Skill Analysis
→ Tuple + List + String
10 marks

Q6
Transaction Analysis
→ Tuple + List
10 marks

Q7
Network Security Analysis
→ String + List
10 marks

Q8
Code Analysis and Debugging
→ List + Tuple + String
10 marks

Q9
Word Frequency and Data
Analysis
→ String + List + Tuple
10 marks

Q10
Integrated Student Data
Analysis
→ String + List + Tuple
10 marks

10/10 answered

Q8 Code Analysis and Debugging — List + Tuple + String

10 marks

Consider the following program:

```
records = [  
    ("Arun", "Python"),  
    ("ARUN", "Python"),  
    ("Bala", "Java"),  
    ("bala", "Java"),  
    ("Charan", "Python")  
]  
  
names = []
```

```
for record in records:  
    name = record[0].strip().lower()  
  
    if name not in names:  
        names.append(name)
```

print(names)

Analyze the program:

1. What happens during each iteration?
2. What value is stored in record?
3. What does record[0] represent?
4. Why are strip() and lower() used together?
5. Why does 'Arun' appear only once?
6. Why does 'Bala' appear only once?
7. What is the final output?
8. Modify the logic to find unique employees and their skills.
9. Analyze what happens if record[0] is changed instead.
10. Explain why modifying record[0] directly is not allowed.

Expected Output:

['arun', 'bala', 'charan']

Your Code

```
1 records = [  
2     ("Arun", "python"),  
3     ("arun", "python"),  
4     ("bala", "java"),  
5     ("bala", "java"),  
6     ("Charan", "python")  
7 ]  
8  
9 names = []  
10  
11 for record in records:  
12     name = record[0].strip().lower()  
13     if name not in names:  
14         names.append(name)  
15  
16 print(names)
```

Run

Save

Input

```
records = [  
    ("Arun", "Python"),  
    ("ARUN", "Python"),  
    ("Bala", "Java"),
```

Output

['arun', 'bala', 'charan']

CO4: AT2: CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks

Q2
Customer Type Analysis
— String + List
10 marks

Q3
Product Inventory Analysis
— List + Tuple
10 marks

Q4
Text Analysis
— String + List
10 marks

Q5
Employee Skill Analysis
— Tuple + List + String
10 marks

Q6
Transaction Analysis
— Tuple + List
10 marks

Q7
Password Security Analysis
— String + List
10 marks

Q8
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9
Word Frequency and Data
Analysis
— String + List +
Tuple
10 marks

Q10
Integrated Student Data
Analysis
— String + List +
Tuple
10 marks

10/10 answered

Q9: Word Frequency and Data Analysis — String + List + Tuple

10 marks

Consider:

text = "data science uses data analysis and data visualization"

Analyze the string using Python.

Tasks:

1. Convert the sentence into a list of words.
2. Count the total words.
3. Find unique words.
4. Find repeated words.
5. Count the frequency of every word.
6. Identify the most frequently occurring word.
7. Create tuples of the form:
(word, frequency)
8. Sort the tuples based on frequency.
9. Find words having frequency greater than 1.
10. Identify the longest word.
11. Find words beginning with 'd'.
12. Calculate the percentage of repeated words.

Expected Output:

Total Words: 9

Unique Words:

['data', 'science', 'uses', 'analysis', 'and', 'visualization']

Frequency:

data → 1

science → 1

uses → 1

analysis → 1

and → 1

visualization → 1

Most Frequent:

data

Repeated Words:

data

Longest Word:

visualization

Your Code

```
1 text = "data science uses data analysis and data visualization"
2
3 words = text.split()
4
5 total_words = len(words)
6
7 unique_words = []
8 for w in words:
9     if w not in unique_words:
10         unique_words.append(w)
11
12 repeated_words = [w for w in unique_words if words.count(w) > 1]
13
14 word_frequencies = {}
15 for word in words:
16     word_frequencies[word] = word_frequencies.get(word, 0) + 1
17
18 most_frequent = max(word_frequencies, key=word_frequencies.get)
19
20 freq_tuples = list(word_frequencies.items())
21
22 sorted_tuples = sorted(freq_tuples, key=lambda x: x[1], reverse=True)
23
24 words_with_high_freq = [word for word, freq in word_frequencies.items() if freq > 1]
25
26 longest_word = max(words, key=len)
27
28 words_starting_with_d = [word for word in words if word.startswith('d')]
29
30 if len(unique_words) > 0:
31     percentage_repeated = (len(repeated_words) / len(unique_words)) * 100
32 else:
33     percentage_repeated = 0
34
35 print("Total words: %d" % total_words)
36
37 print("Unique words:")
38 print(unique_words)
39 print()
40
41 print("Frequency:")
42 for word, freq in word_frequencies.items():
43     print(f"{word} → {freq}")
```

Input

"data science uses data analysis and
data visualization"

Output

```
total words: 9
unique words:
['data', 'science', 'uses',
'analysis', 'and',
'visualization']
```

Failed At Least One:

bala

divya

Highest Total:

arun kumar - 274

Lowest Total:

bala - 164

Average > 75:

arun kumar

arun kumar

charan

Duplicate Name:

arun kumar

top 2:

arun kumar - 91.33

arun kumar - 85.83

Subject-wise Highest:

Subject 1 - 88

Subject 2 - 91

Subject 3 - 95

Subject-wise Average:

Subject 1 - 87.60

Subject 2 - 73.40

Subject 3 - 78.40

Your Code

```
1 students = [
2     ("A001", "Arun Kumar", [78, 85, 92]),
3     ("A002", "bala", [45, 67, 32]),
4     ("A003", "arun kumar", [88, 91, 95]),
5     ("A004", "Charan", [74, 76, 91]),
6     ("A005", "divya", [55, 40, 88])
7 ]
8
9 normalized_data = []
10 for eid, name, marks in students:
11     norm_id = eid.strip().upper()
12     norm_name = name.strip().lower()
13     normalized_data.append((norm_id, norm_name, marks))
14
15 print("\nnormalized records:\n")
16 for item in normalized_data:
17     print(f"{item[0]} {item[1]} {item[2]}")
18
19 totals = []
20 averages = []
21 for item in normalized_data:
22     name = item[1]
23     m_list = item[2]
24     total = sum(m_list)
25     avg = total / len(m_list)
26     totals.append((name, total))
27     averages.append((name, avg))
28
29 print("\ntotals:")
30 for name, tot in totals:
31     print(f"{name} -> {tot}")
32
33 print("\naverages:")
34 for name, avg in averages:
35     print(f"{name} -> {avg:.2f}")
36
37 print("\npassed All Subjects:")
38 for item in normalized_data:
39     if all(m >= 50 for m in item[2]):
40         print(item[1])
41
42 print("\nfailed At least One:")
43 for item in normalized_data:
44     if any(m < 50 for m in item[2]):
45         print(item[1])
46
47 highest_tot_tuple = max(totals, key=lambda x: x[1])
48 print(f"\nhighest total:\n{highest_tot_tuple[0]} -> {highest_tot_tuple[1]}")
49
50 lowest_tot_tuple = min(totals, key=lambda x: x[1])
51 print(f"\nlowest total:\n{lowest_tot_tuple[0]} -> {lowest_tot_tuple[1]}")
52
53 print("\naverage > 75:")
54 for name, avg in averages:
55     if avg > 75:
56         print(name)
57
58 print("\nduplicate name:")
59 names_only = [item[1] for item in normalized_data]
60 seen = []
61 duplicates = []
```

Input

stdin input

Output

```
normalized records:
A001 arun kumar [78, 85, 92]
A002 bala [45, 67, 32]
A003 arun kumar [88, 91, 95]
A004 charan [74, 76, 91]
```